

서민재

BE Engineering · Reliability · Search & Data · AI Engineering

Java와 Kotlin, Spring으로 이벤트·배치·검색·캐시 시스템을 설계하고 운영해 왔습니다. **실패 지점과 재시작 기준**을 코드에 남기는 백엔드 설계를 바탕으로, 코드 탐색·설계 검토·테스트를 **에이전트와 스킬**로 구조화합니다.

ckdekrn88@gmail.com

github.com/ch4570

velog.io/@ch4570

linkedin.com/in/devseorex

이력서

포트폴리오

Seoul, Korea

경력 요약

01 · RELIABILITY

실패 지점과 재시작 기준을 설계합니다.

이벤트 상태, 복구 전략, 배치 순서를 코드와 테스트로 관리했습니다.

02 · SEARCH & DATA

재사용할 때와 다시 계산할 때를 나눕니다.

후보·랭킹·캐시를 분리해 필요한 범위만 다시 계산했습니다.

03 · AI-NATIVE

개발 판단을 팀이 다시 쓰는 자산으로 만듭니다.

에이전트·스킬·평가 계약을 관리하고 변경 전에는 사람이 승인합니다.

9종 · 6상태 · 8전략

포인트 이벤트와 15분 정체 알림

10개 테이블

원장 아카이브 이관 순서

9개 소스 · 500→100→20

검색 후보와 랭킹 단계

29개 · 43개

공통 모듈 구현 파일과 테스트 소스

2025.06 — 현재 **웍스피어(유)** JOBKOREA / Albamon Platform · Backend Engineer2024.06 — 2025.04 **토스랩** JANDI · Backend Engineer2023.09 — 2024.05 **오케스트로** IDP / Vista · Backend Engineer2022.11 — 2023.02 **즐거운** Backend Engineer

웍스피어(유) · Backend Engineer

JOBKOREA·Albamon Platform의 이벤트 신뢰성, 배치, 검색·캐시, 공통 테스트 기반을 설계·구현합니다.

2025.06 — 현재

POINT PLATFORM · EVENT RELIABILITY

이벤트 실패 지점을 6개 상태로 남기고 8개 복구 전략을 연결했습니다.

발행 실패와 소비 중 실패는 다시 시작할 위치가 다르므로 처리 단계 자체를 이벤트 레코드에 남겼습니다.

- 내부 이벤트 9종·6개 상태로 생성부터 복구 불가 실패까지 기록했습니다.
- 8개 수동 복구 전략으로 저장된 payload를 이벤트 유형별 처리 경로에 전달했습니다.
- 15분 정체 알림으로 오래 멈춘 이벤트를 유형별로 모아 Slack에 알렸습니다.

운영 기준 전송 실패, 처리 실패, 장기 미처리를 구분해 멈춘 위치부터 복구합니다.

POINT LEDGER · BATCH / ARCHIVE

포인트 소멸을 페이지 단위로 바꾸고 10개 테이블의 이관 순서를 고정했습니다.

- Tasklet을 페이지 기반 Chunk 처리로 바꿔 조회와 쓰기 단위를 나눴습니다.
- point_core 6개·point_service 4개, 총 10개 테이블을 정해진 순서로 이관했습니다.
- 정합성 테스트로 만료 알림 발행 순서와 동시 차감·유료/무료 원장 혼합 경로를 확인했습니다.

재시작 기준 소멸은 페이지, 만료 안내는 발송 상태, 아카이브는 테이블 순서로 처리 위치를 확인합니다.

웍스피어 · BACKEND ENGINEERING

검색·캐시와 테스트 기반

FEED & SEARCH · SERVING / RETRIEVAL

9개 소스를 병렬 조회하고 캐시 재사용·무효화 기준을 분리했습니다.

정상 스크롤은 저장된 후보를 쓰고, 좌표·필터·로그인 주체가 바뀔 때만 필요한 단계부터 다시 계산했습니다.

- 500 → 100 → 20 순으로 9개 OpenSearch 소스의 후보를 랭킹했습니다.
- 30분 세션 캐시를 Snapshot·1차·2차 랭킹 풀로 나눠 재사용 범위를 정했습니다.
- 로그인 전환과 Redis 장애를 별도 분기로 처리하고 중복 제거와 10km 정책을 적용했습니다.

판단 기준 스크롤, 조건 변경, 로그인 전환, Redis 장애를 같은 cache miss로 취급하지 않았습니다.

ENGINEERING PLATFORM · TEST SUPPORT

29개 구현 파일과 43개 테스트 소스로 공통 테스트 기반을 만들었습니다.

fixture, mock, WebMvc, JPA 준비 규칙을 공통 도우미로 묶고 도우미 자체도 테스트했습니다.

- 29개 구현 파일에 fixture·mock·WebMvc·JPA·transaction 준비를 정리했습니다.
- 4가지 테스트 진입점을 나눠 필요한 Spring 컨텍스트만 선택하게 했습니다.
- 43개 테스트 소스로 공통 동작을 확인하고 기본 profile과 JavaDoc을 정리했습니다.

Agent Lab · 개발 판단을 설치·평가 가능한 자산으로 만들었습니다.

코드 탐색·설계 검토·테스트·리뷰 절차를 다른 프로젝트에서도 다시 쓸 수 있는 단위로 정리했습니다.

V0.1.0.0

역할·지식·절차·평가를 나눠 manifest로 연결했습니다.

- 15개 에이전트·14개 지식 번들·32개 스킬과 24개 연결, 11개 의존성을 선언했습니다.
- 설치 회귀 52/52·Codex 탐색 32/32로 자산과 의존성이 공식 경로에 설치되는지 확인했습니다.
- 구조 7/7·행동 계약 10/10을 정적으로 검증하고 사람이 승인한 변경만 다음 설치에 반영합니다.

현재 경계 실행 런타임이 아닌 이식 가능한 자산 저장소입니다. 장기 메모리, 프롬프트 자동 수정, 스킬 자동 승격은 포함하지 않았습니다.

PREVIOUS ROLE

토스랩 · JANDI Backend Engineer

2024.06 — 2025.04

커버리지 70%에도 배포가 불안해 실제 구현 검증과 CI 재현 조건부터 다시 봤습니다.

- Spring Boot 2.3.8 → 3.3.3 전환과 함께 테스트를 0개에서 160개 이상으로 늘렸습니다.
- 약 200만 건 데이터 보정을 위해 JPA 변경 감지 오류를 실제 구현 테스트로 재현했습니다.
- 자원 증설 없이 CI 실행·컨텍스트 재사용·초기화 구조를 바꿔 메모리 부족을 해결했습니다.

이전 경력

오케스트로 · Backend Engineer

IDP / Vista

2023.09 — 2024.05

- API Gateway·Security·JWT 기반 인증·인가와 SonarQube·Jenkins 연동 API를 개발했습니다.
- VM 메트릭·OpenSearch 조회 API와 조건별 동적 쿼리를 구현했습니다.

즐거운 · Backend Engineer

자사 서비스 개발·운영

2022.11 — 2023.02

자사 백엔드 기능을 개발하고 운영 이슈를 대응했습니다.